Graphic Era deemed to be University
DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

ForceForge: An Interactive 2D Physics Simulation Sandbox

Team Name: *Team #*	*Team Jkd* *DSCPP-III-2026-T098*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Negi, Pranjal – 2510370602* *2510370602@gau.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Rawat, Dilasha – 2510012530* *2510012530@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Jakhmola, Rishabh – 25101210555* *25101210555@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Chhabra, Akshit – 2510011905* *2510011905@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

ForceForge is an open and educational 2D Newtonian physics sandbox built with C++17 and SFML. In textbooks mechanics is taught through equations and static diagrams.. Forceforge lets students watch gravity, momentum, collisions and projectile motion unfold in real time. They can change the parameters. See what happens. Users can fire projectiles from a launcher. Simply click to spawn circles, triangles and squares. They can set the mass, speed and angle for each body. They can select any body to edit it.
Four presets, in ForceForge show physics concepts. One shows projectile motion. Another shows collisions. A third shows collisions. The fourth shows fall. In one demo three bodies of mass land together to prove that acceleration does not depend on mass.
ForceForge also demonstrates C++ object-oriented design. It uses STL containers. It includes partitioning. It features based collision response. All of this is built to help students learn physics by seeing it in action.

Updated Project Approach and Architecture

The application is written in C++17. It uses SFML 3.0. The program deals with drawing the window and handling user input. There is no network part. It runs as a process on a desktop. It doesn't use any data protocol. A World class manages the scene. It moves the scene in fixed steps of 1/60 seconds. During each step forces are collected. The integrator updates speed and location. Then collisions are handled.
Collisions are managed using a hash grid for the stage. After that specific tests are done for circle, box and convex-polygon pairs. An impulse resolver is used. This resolver includes bounce, friction, position correction and a way to handle objects that are just resting against each other.
Forces are handled as types. There's a ForceGenerator class. It is extended by GravityForce, DragForce and SpringForce. The scene information is stored in data types. This makes it simple to undo, redo, save and load by copying. User actions go through an EventQueue.
Change, from the plan: originally we planned a PhysicsObject with Particle, Circle and Box as subtypes. Instead we created a Body data type with a ShapeType (Circle, Box, Polygon). Using data types makes undo, redo and saving easier. Polymorphism is shown through ForceGenerator. The first stage uses a grid. A quadtree is planned for Phase 3.

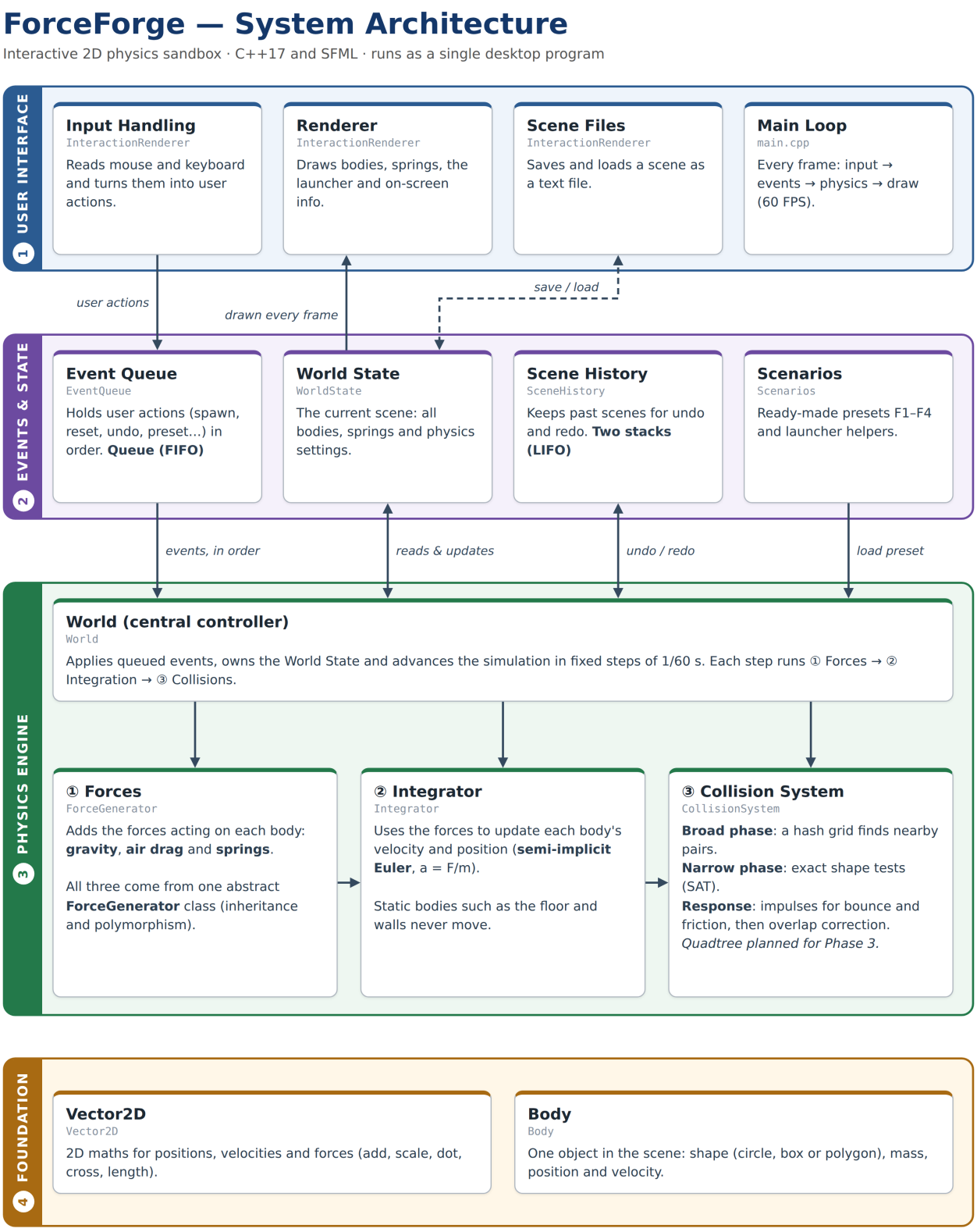


Figure: ForceForge system architecture

Tasks Completed

Task Completed	Team Member
Vector2D math operations include length, normalise, dot product and cross product. There is a Body data type that supports circle, box and polygon shapes. The World class handles a fixed time step of 1/60 seconds. It allows body picking and dragging and manages the simulation loop.	Akshit Chhabra
The application uses SFML for window creation, rendering and input handling. A launcher lets users adjust mass, speed and angle. It supports click-and-drag selection shows on-screen info. Displays a controls list. It includes scene save and load features. The main entry point is main.cpp. It works with both SFML 2.6 and 3.	Rishabh Jakhmola
Narrow-phase collision detection tests include circle-circle, circle-box, box-box and SAT-based polygon tests. The broad phase uses a hash grid. The CollisionSystem::step method runs the collision pipeline. There are presets F1 to F4: projectile, elastic, inelastic and free fall. The launcher comes with helper functions for setup.	Pranjal Negi
Force generators such as gravity, drag and spring forces are implemented through a ForceGenerator class. The simulation uses semiimplicit and explicit Euler integration methods. Impulse-based responses handle restitution, friction, position correction and sleep detection. The system supports undo and redo with stacks. There is also an event queue, for managing simulation events.	Dilasha Rawat
Automated test suite has 123 checks spread across 15 sections and all of them are passing. README file is present. VS Code build task, run task and test task are all set up.	Whole team

Challenges/Roadblocks

• Resting contact: when bounce is turned on bodies that are stacked or lying still must stop moving of bouncing forever. The solver uses a rebound limit; slow impacts get no bounce. It also puts bodies to sleep when they slow down and runs passes each step. Phase three will compare this to measured rebound values.

• Large static bodies in the grid: if a full-width floor is stored in one cell the system cannot find it. Now each body is inserted into every cell that its bounding box touches.

• Keeping four members modules each person owns a part—object model, collision, forces and state interface—so header files and source files must stay in sync. After every merge we rebuild the code. Run the entire test suite.

• SFML 2.6 and SFML 3 have event and text APIs. A compile-time switch on the SFML version lets one code base work with both.

• Click versus drag: selecting a body and moving it must be distinguished. A press turns into a drag only after the mouse moves a pixels. A simple click then selects the body without moving it.

• Educational clarity: many shapes and controls distract from the physics. We trimmed the UI to three shapes and four presets. The controls list can now be hidden.

Tasks Pending

Task Pending	**Team Member (to complete the task)**
Quadtree broad phase together with the grid; run tests for Quadtree, force, grid and Quadtree using 50, 200 and 500 bodies.	Pranjal Negi
Friction demonstration setup; run a friction stress test with moving bodies and fix tunneling issues.	Pranjal Negi
Restitution per body with a combine rule; tests that examine restitution check momentum. Check energy.	Dilasha Rawat
Link scene. Load to keys in the user interface (the functions are and have been tested but are not available, through the UI).	Rishabh Jakhmola
Complete code. Prepare the project demonstration complete the project demonstration.	Whole team

Project Outcome/Deliverables

1. Physics Simulation Application That Works: A tool made with C++17 and SFML where students can build and send objects into the air to learn about physics ideas like gravity, hits, movement and things that fly through the air. The program has a simulation system that uses math to move things ways to show gravity, air resistance and springs and deals, with hits by using pushes.

2. All Parts of the Project Are Ready: The project has ways to split up space with grids and quadtree methods to make hitting things. It has a user area built with SFML, examples that are already made so you can test it quickly and a way to store and bring back scenes. It also has tests that run on their own all the C++ code, clear project papers and a show that explains how the simulator works.

Progress Overview

The core of the project is working: the object model, simulation loop, forces, integration, collision detection and response, presets, interface and tests are in place, and the test suite passed fully at the last run. Roughly three-quarters of the planned work is complete by our estimate, which covers Month 1 and almost all of Month 2 of the plan.

Behind schedule: the quadtree (the proposal named a grid or quadtree, and only the grid exists), the friction demonstration preset, and performance testing at a few hundred bodies.

Ahead of schedule: undo/redo, the editable launcher, and the automated test suite were not in the original plan.

Codebase Information

Repository: `https://github.com/Pranjal-ext/ForgeForce`
Branch: main
Structure: include/ (11 headers), src/ (10 source files including main.cpp), tests/test_main.cpp (automated tests), .vscode/tasks.json (build, run and test tasks), README.md. About 3,400 lines of C++ in total, including tests.
Build: C++17 with g++ 9 or newer, linked against SFML 2.6 or SFML 3. The tests build from the same sources with tests/test_main.cpp in place of src/main.cpp.

Important commits:

Pranjal Negi: collision detection and presets. Added the narrow-phase tests, the spatial hash grid, CollisionSystem::step, and presets F1 to F4 with the launcher helpers.

Dilasha Rawat: forces, integration and state. Added gravity, drag and spring forces, the Euler integrator, impulse response, the undo/redo stacks and the event queue.

Rishabh Jakhmola: interface. Added the SFML window, rendering, input handling, the launcher UI, scene save/load and main.cpp.

Whole team: tests and docs. Added test_main.cpp with 123 checks, the README and the VS Code tasks.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Total: 123 automated checks, 123 passed, 0 failed.		
Unit: Vector2D and Body	Pass	13 checks: vector operations, mass and inverse mass static bodies, polygon vertices and point tests.
Unit: Collision detection	Pass	12 checks: circle, box, triangle and hexagon pairs (SAT). Near misses, direction no NaN for identical positions.
Unit: Collision response	Pass	3 checks: mass elastic collision swaps velocities momentum conserved static bodies never move.
Unit: Forces	Pass	12 checks: F equals m g drag opposes motion spring F equals k x with opposite ends dispatch through the base class.
Unit: Integration	Pass	5 checks: v equals a t forces cleared each step; semi-implicit Euler keeps spring energy bounded, Euler gains energy.
Accuracy: Projectile range	Pass	Range versus v squared sine 2 theta over g: 1.7 percent error at 60 steps per 0.4 percent at 240 steps per second.
Accuracy: Spring period	Pass	Measured period within 2 percent of 2 pi root m over k.
Integration: World and simulation behaviour	Pass	23 checks: fixed-step timing all shapes land and rest on the floor bounce then settle, stack, friction stops sliding walls contain fast bodies.
Performance: 100 bodies	Pass	No NaN no escapes physics step under the 8 milliseconds limit. [Add milliseconds per step measured on your machine]
Unit: Event queue and undo/redo stacks	Pass	16 checks: FIFO order, LIFO undo, capacity limit redo cleared by an action reset can be undone.
Functional: Presets	Pass	11 checks: all four load; elastic and inelastic results match theory; inelastic momentum conserved (1200); free-fall bodies of mass fall together.

Test Type	Status (Pass/Fail)	Notes
Functional: Launcher and scene files	Pass	25 checks: angle, speed and mass limits; 20 shots at speed stay, in the arena; save load round trip; damaged or missing files rejected.
Performance: 50/200/500 bodies, grid vs quadtree	Pending	Planned for Phase 3 with the quadtree.

Deliverables Progress

Deliverable	Status	Remarks
C++ source code and object model	Completed	Body data type with ShapeType replaces the planned PhysicsObject hierarchy.
Simulation and numerical integration engine	Completed	Fixed 1/60 s step, semi-implicit Euler.
Force generation (gravity, drag, spring)	Completed	Friction is applied as a contact impulse.
Collision system with impulse response	In progress	Working; per-body restitution added in Phase 3.
Spatial partitioning	In progress	Grid done; quadtree pending.
SFML interface and rendering	Completed	
Preset demonstrations	In progress	Four presets done; friction preset pending.
Scene save/load	In progress	Implemented and tested; not yet bound to a key.
Automated test suite	Completed	123/123 checks pass.
Documentation and final demonstration	Pending	README done.